

Meru मेरु

A private AI workspace on machines your company manages.

For a CTO or CIO weighing a pilot, and for a platform team weighing a managed offering. Facts, measurements and gaps, as of this release.

v0.7.0 · November 2026 · aarora79.github.io/meru

github.com/aarora79/meru
Scan for the code

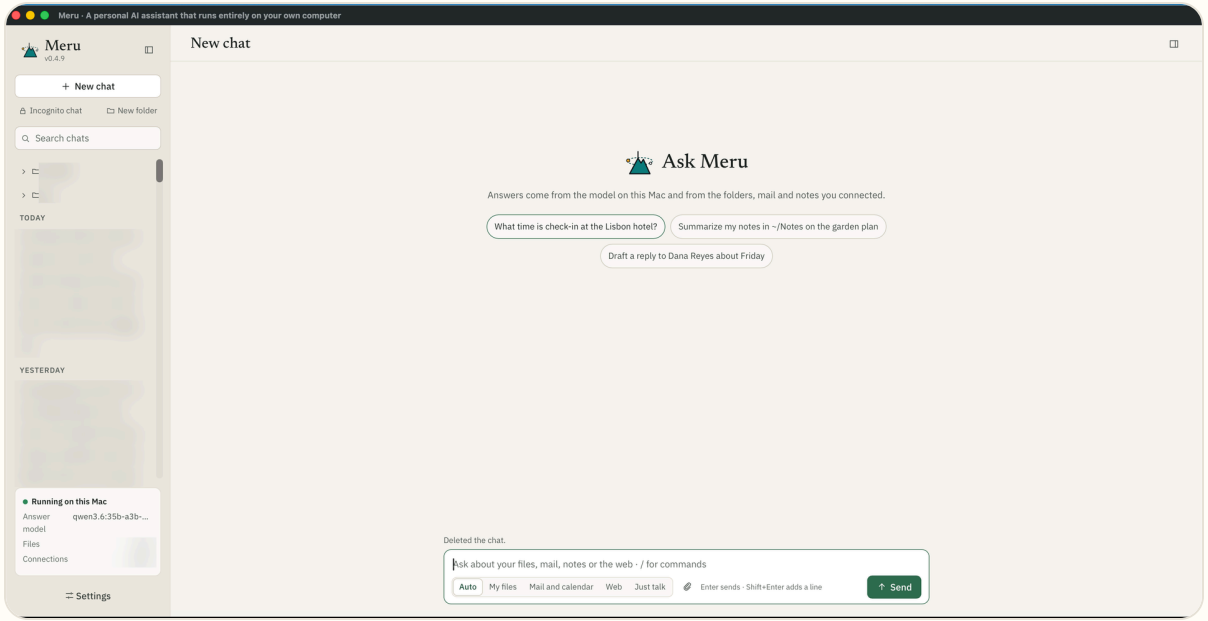
THE QUESTION

The data AI helps with most is the data a company least wants to send out

- **A hosted assistant reads each question on its provider's servers**, with the documents, mail and records attached to it.
- **Each one is a data processor** to assess, contract with and monitor, and its terms can change.
- **Meru takes the other path.** Open-weight models run on the employee's own machine, beside the data, and no model provider is in the path.

WHAT PEOPLE USE

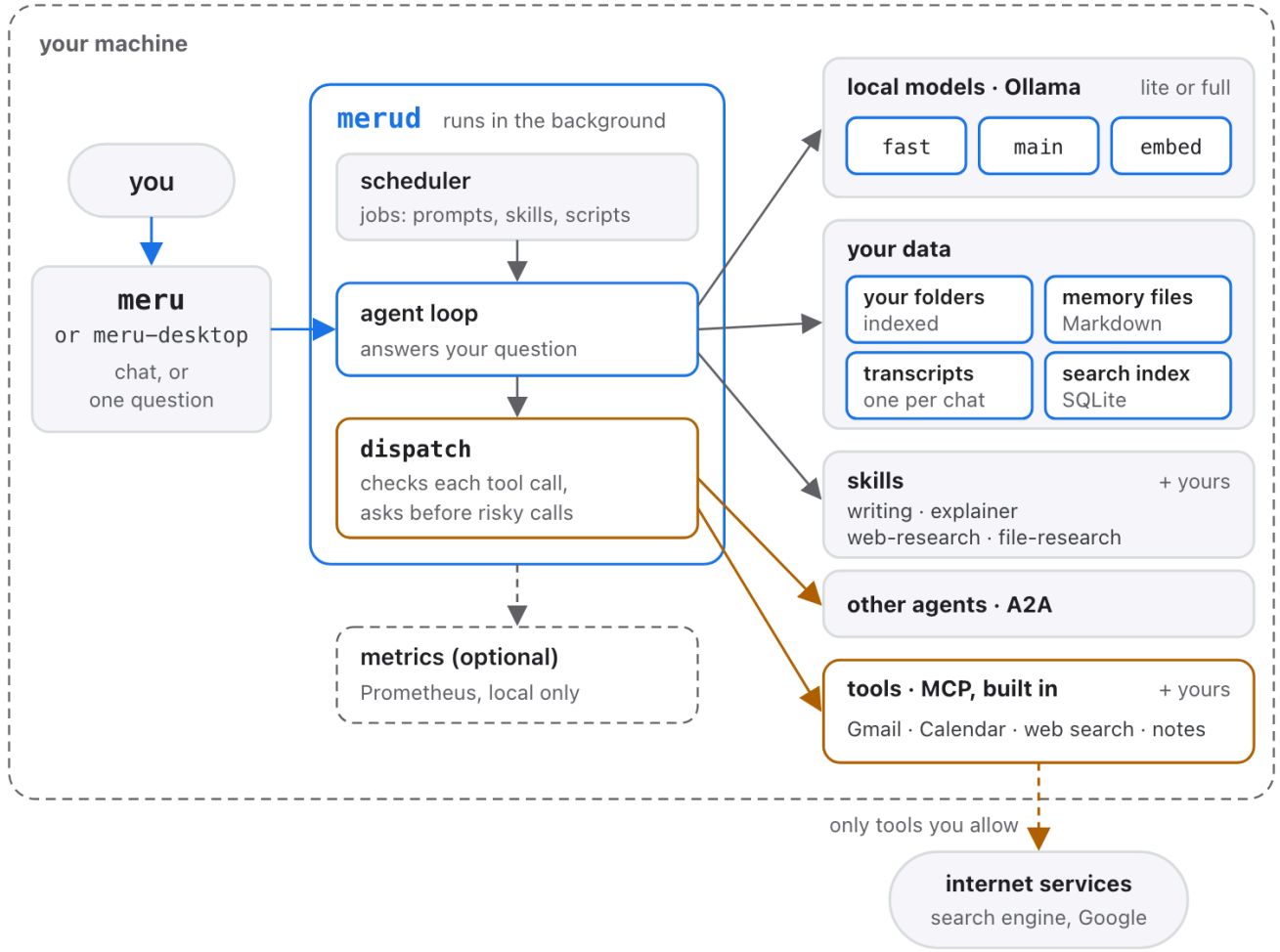
Meru.app, a desktop app on each person's Mac



- **Ask across work data:** documents, mail and calendar, the web, and company systems such as SharePoint, Confluence or Salesforce through MCP servers the company allows.
- **Attach files and pictures,** or speak the question.
- **Approve on a card** any tool call that sends, deletes or changes, before it runs.
- **Settings** for connections, folders, skills and models, and a Usage screen.

The app talks to `merud`, the program that runs the models through a local model server on 127.0.0.1, Ollama today, and owns the data; `meru chat` in a terminal and `meru run --json` for scripts talk to the same `merud`. No question goes to the internet, and none leaves the machine unless the company names a model on its own network and the user says yes; a test in CI fails the build on a provider's SDK or address. Apache License 2.0, written in Go, v0.7.0 in November, six weeks after the first commit on 23 September 2026. macOS on Apple silicon first; Linux supported; Windows untested.

THE BOUNDARY

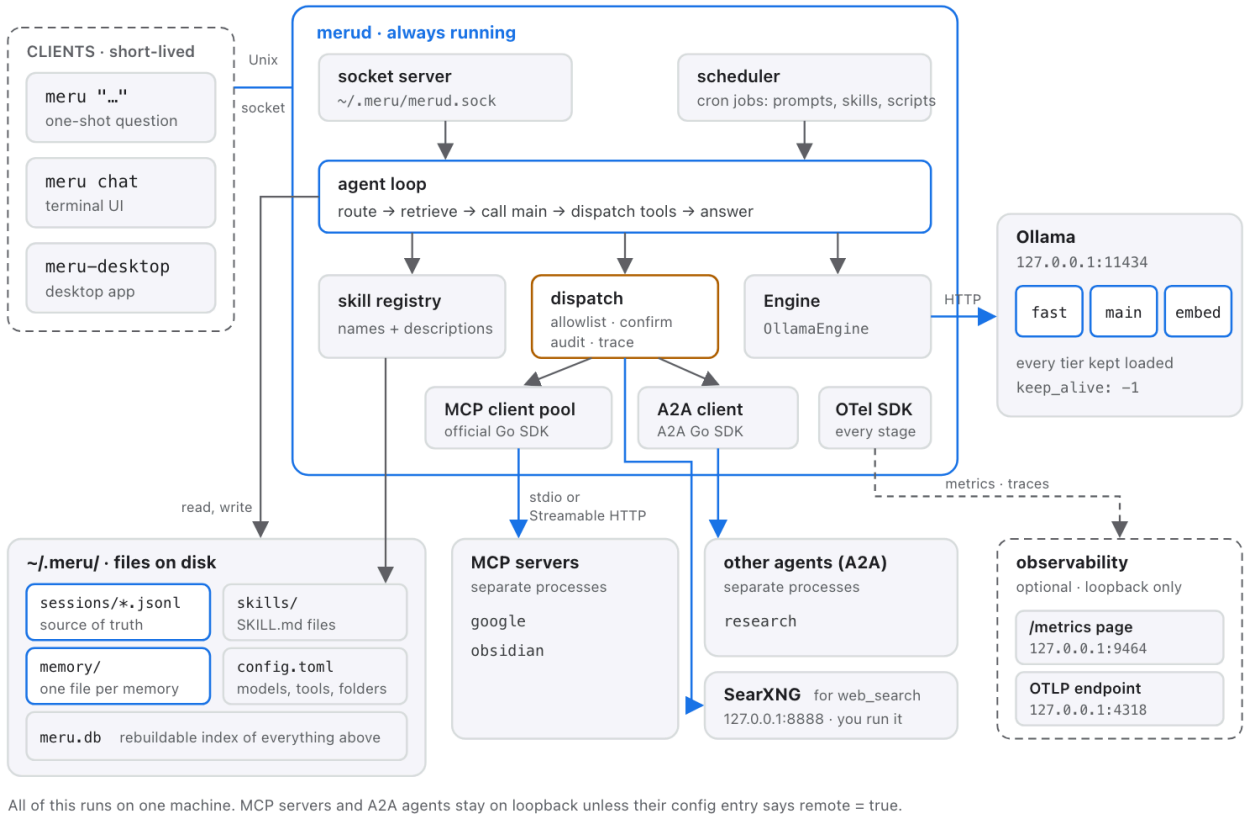


Everything inside the dashed line runs on the employee's machine

The models, the search index, the chat history and the memories stay there. Only tools the configuration allows reach outside.

ARCHITECTURE

Inside `merud`: two ways out, each one place in the code



- **Clients start, ask and exit.** `merud` runs all day and listens only on a Unix socket in `~/.meru`.
- **To models: the Engine,** to Ollama on `127.0.0.1:11434`, with every model kept loaded.
- **To tools:** `dispatch`, to MCP servers, A2A agents, declared commands and SearXNG. A server off the machine needs `remote = true`.
- **Files are the record.** `meru.db`, mode 0600, rebuilds from them.
- **Metrics and traces** go only to 127.0.0.1; `merud` refuses any other address.

Not drawn: the connector supervisors, and the headless Chrome behind a proxy in `merud` for pages that need JavaScript.

ARCHITECTURE

One question, end to end, and the check at each step

Step	What `merud` does	Limits and checks
Ask	The client sends one JSON line over the socket: the question, its scope, any pictures.	Local socket only; 4 MiB per message.
Record	Appends the question to the session's JSON Lines transcript.	An incognito chat stays in memory and writes no file.
Route	The fast model writes one letter, A to D; its log probabilities give the confidence.	Below 0.45, Meru searches and allows tools anyway. About 30 ms.
Search	Keyword (BM25) and vector search, 50 each, merged by reciprocal-rank fusion; memories and past-chat summaries too.	Listed folders only; secrets, hidden files and symlinks skipped. Top 10 go in, numbered for citation.
Prompt	Builds the prompt within a budget, stable parts first so the model server reuses its cache.	History: the last ten turns, cut to 8,000 characters.
Rounds	The answer model asks for tools; `merud` runs each round's calls side by side.	At most 8 rounds, the last with no tools; 8,192 tokens per model call; 5 minutes per turn.
`dispatch`	Checks the allowlist and the scope, asks for approval when config says so, runs the call, logs it.	No wildcards; 60 s per call; secrets stripped; the model reads 16,000 characters of a result, the log keeps 4,000.
Answer	Streams to the client, checks for claims no tool backs, appends the answer with its token counts and trace ID.	A notice under any answer that claims an action no tool took.

DATA FLOW

What leaves the machine, and what doesn't

Data	Where it goes
Questions and answers	Nowhere, by default: the models run on the same machine. An answer model the company names on its own network gets a question only after the user says yes; none goes to the internet.
Files and documents	Indexed into a SQLite file on the machine; read live, never copied out.
Telemetry, crash reports, update checks	None. Metrics go to a Prometheus the company runs on the same machine.
Web searches	The search words go to the engines the company's own SearXNG asks.
Web pages	Public addresses only. Meru asks first for an address no search or question gave, unless the user trusted that website.
Mail and calendar	Read through the user's own account, when they connect it; Google today.
Company systems	SharePoint, Confluence, Salesforce and the like, through an MCP server the company allows, tool by tool. Meru reaches that server and nothing else.

CONTROLS

Controls a security team can check

Control	How Meru does it
Tools	Deny by default. A new tool server contributes nothing until config names its tools. Each tool is Off, Ask or Allow; sending, deleting or changing asks every time.
Network	Connects only to its own machine, except to servers marked remote and to public pages it reads; refuses private addresses at connect time.
Secrets	API keys in one file only the user can read; kept out of the model, the logs and the history.
Supply chain	Every program Meru installs is pinned by version and SHA-256 or image digest; CI signs a record of how its speech server was built.
Enforcement	Policy tests in CI fail the build on a provider's SDK or address, a second path to a tool, or a forbidden import.

AUDIT

Every tool call leaves a record on the machine

- **One path for every tool call.** MCP servers, other agents, local commands and built-in tools all go through `dispatch`, which checks the allowlist, asks for approval, and logs the call.
- **Two records.** A `tool_calls` row with the tool, time, duration, outcome and approval, and the session's transcript, a plain JSON Lines file.
- **Incognito keeps the audit row.** An incognito chat keeps no transcript, and its rows keep no arguments or results, but each call is still counted.
- **Metrics and traces** in OpenTelemetry, to a collector on the same machine only. Prompt and answer text stay out of traces unless config turns capture on.

MEASURED

Answer quality and speed by Mac memory

Mac memory	Answer model	Passed	First word, median	Whole answer, median
under 32 GB	`MiniCPM5-2B`	68%	11.6 s	13.1 s
32 to 47 GB	`gemma4:26b-a4b-it-qat`	80%	17.6 s	18.3 s
48 to 63 GB	`qwen3.6:35b`	89%	5.6 s	10.4 s
64 GB or more	`qwen3.6:35b-a3b-mxfp8`	89%	4.1 s	6.5 s

150 results per model set, 3 runs of 50 questions about the author's own files, notes, mail and calendar, which aren't published. Times include routing, search and tool rounds. Your numbers on your data will differ. Source: docs/benchmarks/results.md, 28 September 2026.

SIZING

What the hardware has to hold

Memory, as the model server loaded it

- `qwen3.6:35b` with a 262,144-token window: 28.8 GB
- The same model at 32,768 tokens: 23.4 GB
- `MiniCPM5-2B` , the fast model: 2.8 GB
- The search model: 0.7 GB

What follows

- A 64 GB Apple silicon Mac runs the full set with room to spare.
- On Linux, setup reads an NVIDIA card's memory: an L40S card with 45 GB got `qwen3.6:35b`.
- Under 32 GB, Meru runs the 2B model and passes fewer questions.
- Meru sizes each answer model's window to fit an eighth of memory.
- The answer model wrote 84 to 93 tokens a second on an M4 Max.

ROLLOUT TODAY

What it takes to put Meru on a machine

- **A Mac installer** on a disk image sets up the model server, web search, folders and `merud` in eleven steps in a window. A one-line installer does the same in a terminal, and on Ubuntu a second one adds the desktop app and a systemd service.
- **One config file per person**, `~/.meru/config.toml`: the models, the folders, the tools and what each may do.
- **Updates are manual.** Meru never checks for them, so the company decides when a new version reaches each machine.
- **Files are the source of truth.** The database can be deleted and rebuilt from the transcripts, memories and config.

COMPARED

Meru and Meta's Muse

Both give each person an agent of their own. Muse runs it in Meta's cloud; Meru, on a machine the company controls.

What Meru gives the company

- **No AI processor.** No question or document reaches a model provider, so there is no vendor to assess and no training opt-out to police.
- **The data stays in the company's boundary:** on its Macs, or on GPU machines in its own cloud account. End the machine and the data goes with it.
- **Runs disconnected** once the models are down, for isolated and classified networks.
- **The company picks the model,** pins it by digest and keeps it; it changes when the company says.
- **Deny by default, set in config:** each tool Off, Ask or Allow, and an approval card before any change.
- **An audit row for every tool call** on the machine, and metrics to the company's own collector.
- **Free under Apache 2.0,** with the source open to review. The cost is hardware, not seats.

What Muse offers that Meru doesn't

- Meta's newest models, Muse Spark and Muse Wonder
- Errands across the web: browsing, forms, bookings, and payments through partners such as Stripe
- Apps for phone, web, macOS and glasses
- No hardware to buy or run

FOR A REGULATED COMPANY

- Meta is the data processor; encryption that keeps Meta out is promised for later in 2026
- Consumer and small-business plans, in the US

As of October 2026. Sources: about.fb.com/news/2026/09/introducing-muse-personal-ai-agent; en.wikipedia.org/wiki/Muse_(AI_agent).

COMPARED

Meru and OpenClaw

Both are open source and run on your own machine. Meru narrows what the model can reach; OpenClaw opens the machine to it.

What Meru gives the company

- **No hosted models.** A model on the company's own network gets a question only when config names it and the user says yes.
- **No shell.** The model runs only commands declared whole in config, filling typed blanks.
- **One audited path.** Every tool call goes through `dispatch`: allowlist, approval, a `tool_calls` row.
- **Deny by default, no wildcards.** A tool server adds nothing until config names its tools.
- **Nothing to expose.** `merud` listens only on a local socket; no gateway faces the network.
- **Pinned installs.** Every program it installs is pinned by version and SHA-256 or image digest; no public skill market in the path.
- **A guard on data in links:** a page the model wants that no search or question gave asks first.

What OpenClaw offers that Meru doesn't

- Any model, including hosted ones such as Claude and Codex
- WhatsApp, Slack, Teams, Signal and 20 more chat apps
- Apps for macOS, Windows, Linux, iOS and Android
- A large community: a foundation and 247,000 GitHub stars

FOR A REGULATED COMPANY

- Tools run on the host; the sandbox is optional and off by default
- Skills come from a public registry; Cisco found a top-ranked one that sent data out

As of October 2026. Sources: github.com/openclaw/openclaw; en.wikipedia.org/wiki/OpenClaw; blogs.cisco.com/ai/personal-ai-agents-like-openclaw-are-a-security-nightmare.

GAPS

What Meru does not do today

For a fleet

- No central console to push config or policy
- No single sign-on or directory link
- Ready-made connectors only for Google and web search; SharePoint, Confluence or Salesforce need an MCP server
- No central log collection: metrics and traces stay on the machine by design
- One person per machine; no shared server

For the project

- Pre-alpha, v0.7.0
- One maintainer; the first outside contribution merged in October 2026
- Digests and notifications come in v0.7
- Windows builds, untested

FOR A PLATFORM PROVIDER

What a managed offering around Meru would have to add

Fleet setup and config

Install, update and push `config.toml` with each team's allowlists, as the company's device management already does for other programs.

A model catalog

Open-weight models chosen per hardware tier, pinned by digest and served from a mirror the company controls.

Hardware

Apple silicon Macs or GPU machines with enough memory per person, on desks or in the company's own account; the boundary holds wherever the machine is.

Connectors and audit

MCP servers for the company's own systems, and a collector the company runs on each machine to gather the audit rows Meru keeps there.

The Apache License 2.0 allows this. Meru's core stays local by design: no cloud model, no telemetry, deny by default.

FOR A CTO OR CIO

A four-week pilot with ten people

Week	What happens	What to look at
1	Pick ten people with Apple silicon Macs of 32 GB or more, or Linux machines with an NVIDIA card of 27 GB or more. Install, add their work folders, connect one system.	Install time; the controls table checked against the config
2 and 3	They use it for real work: questions on their documents, mail, company systems and the web.	The Usage screen: questions, routes, answer times, tool calls by outcome
4	Security reviews the `tool_calls` rows and the network rules; the ten say what worked.	Pass or fail against the company's own criteria



Built in the open

The code, the design documents and every decision are in the repository.

We're happy to take contributions: see the **issue list**. Questions and pilots start there too.

github.com/aarora79/meru · aarora79.github.io/meru

github.com/aarora79/meru
Scan for the code